并行程序设计 — 课件与教材补充概念速查

基于课件 01~06 + 教材指定页码，与 ch01~ch08 笔记互为补充。

一、Flynn 分类法（教材 Page 4）

类型	全称	指令流	数据流	例子
SISD	Single Instruction Single Data	1	1	传统串行 CPU
SIMD	Single Instruction Multiple Data	1	N	向量机、SSE/AVX、GPU
MISD	Multiple Instruction Single Data	N	1	流水线结构（有争议）
MIMD	Multiple Instruction Multiple Data	N	N	多核 CPU、分布式系统

MIMD 细分：

```
MIMD
├── 共享内存 Shared Memory
│   ├── CSM / UMA / SMP ── 集中式，统一延迟
│   └── DSM / NUMA ──────── 物理分布但逻辑统一
│       ├── CC-NUMA ── 有缓存一致性
│       └── NC-NUMA ── 无缓存一致性
└── 分布式内存 Distributed Memory
    ├── MPP ── 大规模并行处理，定制网络
    └── COW ── 工作站机群，商用网络
```

二、SIMD 体系结构优势（教材 Page 5）

优势	说明
能效高	每次数据操作仅取一条指令，比 MIMD 更节能
适合移动设备	能效优势使 SIMD 成为手机/平板的理想选择
程序员思维简单	允许以顺序方式思考，编译器/硬件负责并行
一条指令编码 N 个操作	代码紧凑，取指功耗低
表达性强	明确告知硬件：这些操作独立、使用相同功能单元、访问模式已知
可扩展	同一目标代码可在更多通道（lane）的硬件上运行

三、向量体系结构示例（教材 Page 10）

3.1 向量 vs 标量代码

C 代码：

```
for (i = 0; i < 64; i++)
    C[i] = A[i] + B[i];
```

标量汇编 (RV64G)： 循环体 9 条指令 × 64 次迭代 = ~600 条指令

向量汇编 (RV64V)： 5 条指令

```
LI    VLR, 64       # 设置向量长度 = 64
LV    V1, R1        # 加载 A[0..63]
LV    V2, R2        # 加载 B[0..63]
ADDV.D V3, V1, V2   # 64 个 double 同时相加
SV    V3, R3        # 存储 C[0..63]
```

3.2 DAXPY 示例（Y = a·X + Y，8192 个元素）

```
for (i = 0; i < 8192; i++)
    Y[i] = a * X[i] + Y[i];
```

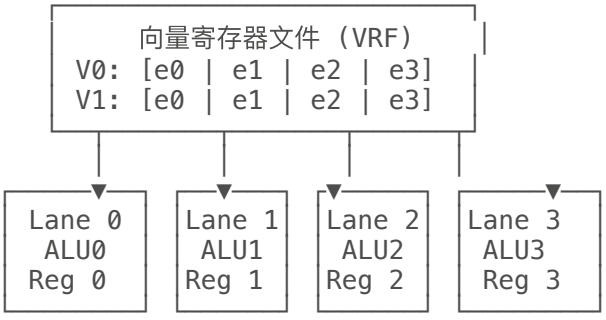
RV64G 标量： ~258 条指令

RV64V 向量（8 条指令）：

```
vsetdcfg 4*FP64       # 启用 4 个双精度向量寄存器
fld f0, a             # 加载标量 a
vld v0, x5            # 加载向量 X
vmul v1, v0, f0       # v1 = a * X (向量–标量乘法)
vld v2, x6            # 加载向量 Y
vadd v3, v1, v2       # v3 = a*X + Y (向量–向量加法)
vst v3, x6            # 存储结果
vdisable              # 禁用向量寄存器
```

若 MVL=64，4 通道 → 8192/(64×4) = 32 轮循环即可完成。

四、四通道向量单元结构（教材 Page 26）



每周期：4 个 Lane 各自处理 1 个元素 → 4 个结果/周期

- **Lane（通道）** = 向量单元中的一条并行数据通路
- 4 lane → 每周期 4 个元素；8 lane → 8 个元素
- 向量寄存器的一行（如 V0 的 4 个元素）映射到 4 个 lane
- 更多 lane = 更高吞吐，但需要更多硬件

五、内存银行 Memory Banks（教材 Page 298-299）

5.1 为什么需要多银行

向量处理器每周期发出大量内存访问请求，单银行无法满足：

```
单银行：一次只能处理一个请求 → 瓶颈
多银行：多个银行独立并行 → 高吞吐
```

5.2 地址到银行的映射

地址按银行**轮流分配**（交叉编址）：

```
8 个银行：
  Bank 0: 地址 0, 8, 16, 24, ...
  Bank 1: 地址 1, 9, 17, 25, ...
  ...
  Bank 7: 地址 7, 15, 23, 31, ...
```

5.3 银行数量计算公式

$$\text{银行数} \approx \frac{\text{处理器数} \times \text{每处理器每周期访问次数} \times \text{SRAM周期}}{\text{CPU周期}}$$

示例： 32 处理器 × 6 次访问/周期 × 15ns / 2.167ns ≈ 1344 个

5.4 银行冲突

无冲突（stride=1）： 连续访问 → 轮流打不同银行 → 全并行

严重冲突（stride=银行数的整数倍）： 所有访问打同一银行 → 排队串行

$$\text{实际使用银行数} = \frac{\text{总银行数}}{\gcd(\text{stride}, \text{总银行数})}$$

六、Roofline 模型（教材 Page 307-310）⭐

6.1 是什么

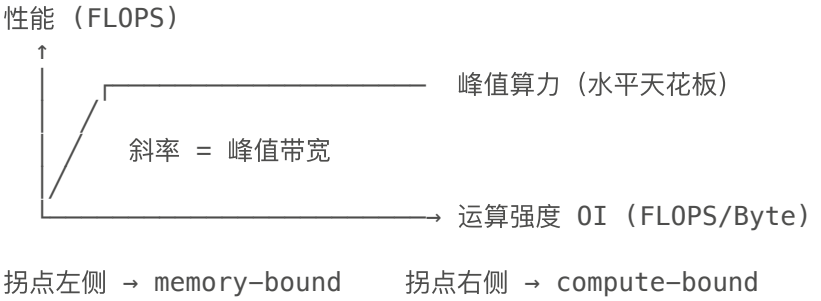
判断内核是 **compute-bound** 还是 **memory-bound** 的可视化模型。

6.2 公式

$$\text{Attainable Performance} = \min(\text{Peak FLOPS},\ \text{Peak Bandwidth} \times \text{OI})$$

$$\text{OI (Operational Intensity)} = \frac{\text{总浮点运算次数}}{\text{总内存访问字节数}} \quad (\text{FLOPS/Byte})$$

6.3 图示



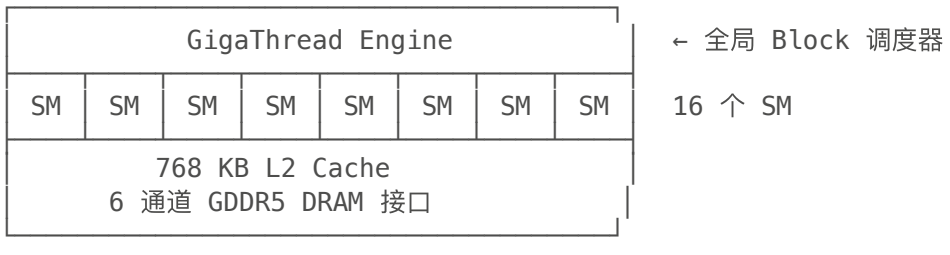
拐点值 = 峰值 FLOPS / 峰值 Bandwidth

6.4 优化方向

区域	瓶颈	优化策略
Memory-bound	内存带宽	合并访问、共享内存、数据复用、减少冗余读写
Compute-bound	算力	增加并行度、使用更快指令、减少控制开销

七、GPU 体系结构（教材 Page 313-320）

7.1 Fermi 架构



每个 SM:
  2 Warp Scheduler + 2 Dispatch Unit
  32 CUDA Core, 16 LD/ST, 4 SFU

```
64KB 可配置片上内存（L1 + Shared Memory）
32768 × 32bit 寄存器
```

7.2 Kepler vs Fermi

特性	Fermi	Kepler
CUDA Core/SM	32	**192**
Warp Scheduler/SM	2	**4**
最大 Warp/SM	48	**64**
最大线程/SM	1536	**2048**
动态并行	❌	✅
Hyper-Q (硬件队列)	1	**32**

7.3 GPU 术语速查（教材 Page 61）

术语	含义
SM	Streaming Multiprocessor，GPU 的计算单元
CUDA Core / SP	基本标量 ALU，执行整数/浮点运算
Warp	32 线程硬件执行组，SM 调度的基本单位
SIMT	单指令多线程：标量编程模型，SIMD 硬件执行
Thread Block	分配到同一 SM 的线程组（最多 1024）
Grid	执行同一 Kernel 的所有 Block
Warp Scheduler	从就绪 Warp 中选择发射的调度器
Dispatch Unit	将指令分发到具体执行单元
LD/ST	加载/存储单元
SFU	Special Function Unit（sin/cos/sqrt/exp）
FMA	Fused Multiply-Add：`d = a × b + c`
Register File	SM 片上寄存器，在活跃 Warp 间分配
Shared Memory	Block 内共享的片上 SRAM
L1 / L2 Cache	SM 私有 / 全局共享缓存
DRAM	片外全局内存 (GDDR5/HBM)

八、CUDA C 基础（课件-01）

8.1 CUDA 是什么（Page 2）

- Compute Unified Device Architecture
- GPU 上的通用并行计算平台
- 扩展 C/C++，使用 `nvcc` 编译 `.cu` 文件

8.2 简单处理流程（Page 8）

```
1. cudaMalloc    → 在 GPU 上分配内存
2. cudaMemcpy    → 从 CPU 复制数据到 GPU
3. Kernel<<<>>> → 在 GPU 上并行执行
4. cudaMemcpy    → 从 GPU 复制结果回 CPU
5. cudaFree      → 释放 GPU 内存
```

8.3 GPU Kernel：设备代码（Page 10-11）

```
// __global__ = 在 Device 执行，从 Host 调用
__global__ void kernel_name(args) { ... }

// 启动语法
kernel_name<<<gridDim, blockDim>>>(args);
```

Kernel 限制： void 返回、只能访问设备内存、不支持递归（CC<3.5）、不支持静态变量、不支持函数指针。

8.4 向量加法：并行执行（Page 12-16）⭐

```
// 主机端串行版本
void vecAdd_cpu(float* A, float* B, float* C, int N) {
    for (int i = 0; i < N; i++)
        C[i] = A[i] + B[i];
}

// 设备端并行版本（每个线程计算一个元素）
__global__ void vecAdd_gpu(float* A, float* B, float* C, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N)
        C[i] = A[i] + B[i];
}

// 主机端调用
int threads = 256;
int blocks  = (N + 255) / 256;          // 向上取整
vecAdd_gpu<<<blocks, threads>>>(d_A, d_B, d_C, N);
```

关键点：

- 循环在每个线程中"消失"了 — 每个线程处理一个数组元素
- `if (i < N)` 边界检查必须写，因为启动的线程数可能超过 N

九、CUDA 共享内存（课件-02, Page 6-13）

9.1 为什么用共享内存

全局内存访问延迟 400-800 周期，共享内存 ~20 周期。**把频繁访问的数据从全局内存搬到共享内存 → 巨大性能提升。**

9.2 使用共享内存

```
__global__ void sharedMemKernel(float* in, float* out, int n) {
    __shared__ float tile[256];          // ★ 片上共享内存

    int tid = threadIdx.x;
    int idx = blockIdx.x * blockDim.x + tid;

    // 协作加载：每个线程加载一个元素到共享内存
    tile[tid] = in[idx];
    __syncthreads();                     // ★ 确保全部加载完

    // 基于共享内存做计算（快）
    out[idx] = tile[tid] * 2.0f;
}
```

9.3 关键规则

规则	说明
一定要同步	`__syncthreads()` 确保所有线程完成共享内存写入后再读取
不要过度使用	共享内存有限（Fermi/Kepler 最多 48KB/Block），用多了 Block 数减少
避免 **Bank Conflict**	同 warp 内多线程同时访问同一 Bank 的不同地址 → 串行化

9.4 共享内存 Bank Conflict

共享内存被分成 32 个 Bank（Fermi+），连续 4 字节映射到连续 Bank：

```
Bank 0: word 0, 32, 64, ...
Bank 1: word 1, 33, 65, ...
...

无冲突：所有线程访问不同 Bank
2-way conflict：2 个线程打同一 Bank → 分 2 次
N-way conflict → 分 N 次（最差）
```

解决方法： 数组加 padding（如 `__shared__ float tile[256+1]`）错开 Bank 映射。

十、CUDA 执行模型（课件-03, Page 6-7）

10.1 执行模型（Page 6）

```
Grid → 分配到 GPU
  └── Block → 分配到 SM
        └── Warp (32 线程) → SM 调度的基本单位
              └── Thread → 最小的执行单元
```

- 一个 Block 只在一个 SM 上执行
- 一个 SM 可同时驻留多个 Block
- Block 被拆分为 Warp，每个 Warp 32 线程
- **Warp 是 SM 调度和执行的基本单位**

10.2 Warp（Page 7）

```
Block 有 256 线程 → ceil(256/32) = 8 个 Warp

Block 的逻辑布局 vs 物理 Warp 分组:
  1D block: threadIdx.x 连续 32 为一组
  2D block: 按 (threadIdx.y * blockDim.x + threadIdx.x) 线性化后 连续 32 为一组
  3D block: 同理，按 x → y → z 顺序线性化
```

Warp 发散：

```
// ❌ 发散：同一 warp 内部分线程走不同分支
if (tid % 2 == 0) { ... }  // 16 线程干活，16 线程闲置 → 串行化

// ✅ 不发文：分支粒度对齐到 warp 大小
if ((tid / 32) % 2 == 0) { ... }  // 整个 warp 走同一路
```

十一、GPU 内存操作（课件-04 Part 2, Page 8-17）

11.1 全局内存访问模式

合并访问 Coalesced Access：

```
✅ 合并：连续线程访问连续地址
  Thread 0 → addr 0, Thread 1 → addr 4, Thread 2 → addr 8 ...
  → 一个 128 字节事务覆盖 32 个线程

❌ 跨步/随机：一次事务只服务少数线程
  Thread 0 → addr 0, Thread 1 → addr 128, Thread 2 → addr 256 ...
  → 需要多个事务，带宽利用率低
```

11.2 只读缓存

Kepler+ 支持用 `__ldg()` intrinsic 通过只读缓存加载数据：

```
float val = __ldg(&global_array[idx]);  // 通过只读缓存加载
```

11.3 共享内存配置

```
// 配置每个 Kernel 的片上内存划分
cudaFuncSetCacheConfig(kernel, cudaFuncCachePreferShared);
// cudaFuncCachePreferShared  → 48KB Shared + 16KB L1
// cudaFuncCachePreferL1      → 16KB Shared + 48KB L1
// cudaFuncCachePreferEqual   → 32KB + 32KB
// cudaFuncCachePreferNone    → 默认
```

十二、GPU Shared Memory 高级用法（课件-04 Part 2, Page 20-25）

12.1 使用 Shared Memory 做矩阵乘法（分块/Tiling）

```
__global__ void matMulTiled(float* A, float* B, float* C, int N) {
    __shared__ float As[TILE][TILE];   // A 的分块
    __shared__ float Bs[TILE][TILE];   // B 的分块

    int row = blockIdx.y * TILE + threadIdx.y;
    int col = blockIdx.x * TILE + threadIdx.x;
    float sum = 0.0f;

    for (int t = 0; t < N/TILE; t++) {
        // 协作加载分块到共享内存
        As[threadIdx.y][threadIdx.x] = A[row * N + t * TILE + threadIdx.x];
        Bs[threadIdx.y][threadIdx.x] = B[(t * TILE + threadIdx.y) * N + col];
        __syncthreads();

        // 在共享内存分块上累加
        for (int k = 0; k < TILE; k++)
            sum += As[threadIdx.y][k] * Bs[k][threadIdx.x];
        __syncthreads();
    }
    C[row * N + col] = sum;
}
```

12.2 Tiling 的好处

- 每个 tile 从全局内存只加载一次
- 被多个线程重复使用（数据复用）
- 对全局内存的访问次数减少 TILE 倍

十三、并行归约优化全链（课件-05, Page 12-33）⭐

13.1 优化演进表

版本	技术	相对加速比	关键改进
V1	原始邻居对	1.0×	基线（高度 warp 发散）
V2	减少发散（索引重排）	1.26×	相邻线程同时干活
V3	交错对（Interleaved）	1.69×	改变内存访问模式
V4	循环展开 ×2	3.42×	每线程处理 2 个数据块
V5	循环展开 ×8	~6×	每线程处理 8 个数据块
V6	Warp 级展开 + volatile	8.65×	消除最后 6 轮循环
V7	完全展开	9.16×	用编译期 if 替代循环
V8	模板函数完全展开	最大	`template<unsigned int>` 编译期全部剥除

13.2 关键代码模式

交错归约（V3）：

```
for (int stride = blockDim.x / 2; stride > 0; stride >>= 1)
    if (tid < stride)
        idata[tid] += idata[tid + stride];
```

Warp 级展开（V6）：

```
if (tid < 32) {
    volatile int* vmem = idata;      // volatile 防编译器重排
    vmem[tid] += vmem[tid + 32];
    vmem[tid] += vmem[tid + 16];
    vmem[tid] += vmem[tid + 8];
    vmem[tid] += vmem[tid + 4];
    vmem[tid] += vmem[tid + 2];
    vmem[tid] += vmem[tid + 1];
}
```

模板完全展开（V8）：

```
template <unsigned int iBlockSize>
__global__ void reduceCompleteUnroll(int* g_idata, int* g_odata, unsigned int n) {
    // iBlockSize 是编译期常量 → 所有 if(iBlockSize>=...) 在编译时求值
    // 不成立的分支直接被编译器删除，零运行时分支开销
}
```

13.3 为什么每一步有效

优化	消除了什么
交错对	减少 Warp 发散
循环展开 ×N	减少循环维护指令 + 每个线程更多独立访问 → 隐藏内存延迟
Warp 展开 + volatile	消除最后 6 轮循环 + 避免编译器将累加放到寄存器导致写回丢失
完全展开 + 模板	消除所有循环分支，编译器常量传播极致优化

十四、统一内存 Unified Memory（课件-06, Page 5-9）

14.1 是什么

CUDA 6+ 引入的单一地址空间，CPU 和 GPU 使用同一个指针，物理数据迁移由驱动自动管理。

```
// 传统方式：显式管理两块内存
float *h_data, *d_data;
h_data = malloc(size);
cudaMalloc(&d_data, size);
cudaMemcpy(d_data, h_data, size, cudaMemcpyHostToDevice);
// ... kernel ...
cudaMemcpy(h_data, d_data, size, cudaMemcpyDeviceToHost);

// 统一内存：一块内存，两边都能访问
float *data;
cudaMallocManaged(&data, size);   // ★ 替代 malloc + cudaMalloc
// ... kernel（直接传 data）...
cudaDeviceSynchronize();           // ★ 等 kernel 完成
// 直接在 CPU 访问 data，不需要 cudaMemcpy!
cudaFree(data);
```

14.2 优势

优势	说明
简化编程	不用手动 `cudaMemcpy`，一个指针两边用
按需迁移	驱动自动在 CPU/GPU 间迁移数据页
减少出错	消除"忘了拷贝"/"拷错了方向"类错误
支持oversubscription	GPU 内存不够时，自动换出到 CPU 内存

14.3 注意事项

- 仍然需要 `cudaDeviceSynchronize()` 确保 Kernel 执行完再在 CPU 读结果
- 页错误驱动的按需迁移可能引入隐式延迟
- 适合渐进式移植和代码原型阶段

十五、GPU 条件分支 / 发散（教材 Page 323-326）⭐

15.1 三种情况

范围	GPU 行为	性能影响
同 warp 内部分线程走不同分支	串行执行两路，屏蔽不参与线程	严重 ❌
整个 warp 条件相同	只执行一路	无影响 ✅
不同 warp 走不同分支	各自独立执行（MIMD 行为）	无影响 ✅

15.2 CUDA 编译器的谓词优化

```
// 原始代码
if (cond)  a = b + c;
else       a = b - c;

// 编译器可能转换为谓词指令：
// p0 = cond;  p1 = !cond;
// a = (p0 ? b+c : 0) + (p1 ? b-d : 0);   // 两路都算但只保留一路
```

优点： 消除分支跳转。缺点： 两路指令都执行，浪费计算。

15.3 避免 Warp 发散的方法

```
// ❌ 发散
if (tid % 2 == 0) { ... }

// ✅ 按 warp 对齐
if ((tid / 32) % 2 == 0) { ... }

// ✅ 根据 blockIdx 分支（不同 block 在不同 SM 上）
if (blockIdx.x % 2 == 0) { ... }
```

快速索引

主题	来源	本文位置
Flynn 分类法	教材 pg 4	§一
SIMD 优势	教材 pg 5	§二
向量架构示例	教材 pg 10	§三
四通道向量单元	教材 pg 26	§四
内存银行	教材 pg 298-299	§五
Roofline 模型	教材 pg 307-310	§六
GPU 架构	教材 pg 313-320	§七
GPU 术语	教材 pg 61	§七.3
CUDA 基础	课件-01 pg 2-16	§八
共享内存	课件-02 pg 6-13	§九
执行模型 + Warp	课件-03 pg 6-7	§十
GPU 内存操作	课件-04 Part2 pg 8-17	§十一

主题	来源	本文位置
Shared Memory 高级	课件-04 Part2 pg 20-25	§十二
并行归约优化全链	课件-05 pg 12-33	§十三
统一内存	课件-06 pg 5-9	§十四
GPU 条件分支	教材 pg 323-326	§十五